REST API Architecture Guide

Core Concepts, Best Practices, Design Standards, and Status Codes

1. Introduction to REST

REST (Representational State Transfer) is an architectural style for designing networked applications. It relies on a stateless, client-server, cacheable communications protocol—standardized around HTTP methods.

- **Client-Server Separation:** The user interface concerns are separated from data storage concerns, improving portability across multiple platforms.
- **Statelessness:** Every HTTP request from client to server must contain all of the information necessary to understand and process the request.
- **Cacheability:** Responses must define themselves as cacheable or non-cacheable to prevent client-side reuse of stale data.

2. Standard HTTP Methods

REST APIs map CRUD operations directly to standard HTTP verbs:

Operation	HTTP Method	Description & Idempotency
Create	`POST`	Creates a new resource. Not idempotent.
Read	`GET`	Retrieves a resource or collection. Safe and idempotent.
Update	`PUT / PATCH`	PUT replaces entirely (idempotent); PATCH applies partial updates.
Delete	`DELETE`	Removes a specified resource. Idempotent.

3. Resource Naming Conventions

API endpoints should be intuitive, hierarchical, and utilize plural nouns for resource collections rather than verbs.

- **Correct:** `GET /api/v1/users` (Fetches all users)
- **Correct:** `GET /api/v1/users/42/orders` (Fetches orders for user 42)
- **Incorrect:** `GET /api/v1/getUsers` or `POST /api/v1/deleteUser/42`

4. HTTP Status Codes

Responses should always return appropriate semantic status codes to indicate success, failure, or client error:

Code	Status Category	Common Usage Example
200 - 204	Success	`200 OK`, `201 Created`, `204 No Content`
400 - 404	Client Error	`400 Bad Request`, `401 Unauthorized`, `404 Not Found`
500	Server Error	`500 Internal Server Error`, `503 Service Unavailable`

5. Sample JSON Response Structure

```
{ "status": "success", "code": 200, "data": { "userId": 42, "name": "Aarav Sharma", "email": "aarav.sharma@example.com", "roles": ["developer", "admin"] }, "timestamp": "2026-10-31T23:04:00Z" }
```